

Lab 1

Objective: To Implement The DDA(Digital Differential Analyzer) Line Drawing Algorithm

Theory

The Digital Differential Analyzer (DDA) line drawing algorithm is a simple and efficient method used in computer graphics to draw a straight line between two given points on a screen. It is an incremental scan-conversion algorithm based on calculating either y or x values at regular intervals between the endpoints. It works by calculating the intermediate points required to draw the line using floating-point arithmetic. The basic idea is to start from one endpoint and generate the next point by incrementally adding fixed values to the current coordinates based on the slope of the line

Algorithm

Step1: Start

Step2: Declare $x_1, y_1, x_2, y_2, dx, dy, x, y$ as integer variables.

Step3: Enter value of x_1, y_1, x_2, y_2 .

Step4: Calculate $dx = x_2 - x_1$

Step5: Calculate $dy = y_2 - y_1$

Step6: If $ABS(dx) > ABS(dy)$

Then $step = abs(dx)$

Else

Step7: $xinc = dx / step$

$yinc = dy / step$

assign $x = x_1$

assign $y = y_1$

Step8: Set pixel (x, y)

Step9: $x = x + xinc$

$y = y + yinc$

Set pixels (Round (x), Round (y))

Step10: Repeat step 9 until $x = x_2$

Step11: End

Program

```
#include <graphics.h>

#include <conio.h>

#include <math.h>

#include <dos.h>

void main() {

    int gd = DETECT, gm;

    float x1, y1, x2, y2, dx, dy, steps, x, y, x_inc, y_inc;

    int i;

    initgraph(&gd, &gm, "C:\\Turboc3\\Projects");

    printf("Enter x1 and y1: ");

    scanf("%f%f", &x1, &y1);

    printf("Enter x2 and y2: ");

    scanf("%f%f", &x2, &y2);

    dx = x2 - x1;

    dy = y2 - y1;

    steps = (abs(dx) > abs(dy)) ? abs(dx) : abs(dy);

    x_inc = dx / steps;

    y_inc = dy / steps;

    x = x1;

    y = y1;

    for (i = 0; i <= steps; i++) {

        putpixel(round(x), round(y), WHITE);

        x += x_inc;

        y += y_inc;

        delay(10);

    }

    getch();

    closegraph();

}
```

Output



Conclusion

The DDA Line Drawing Algorithm provides a simple and efficient way to draw straight lines using incremental calculations. By plotting pixels based on floating-point increments, it accurately generates smooth lines between two points. This experiment successfully demonstrated the practical implementation of the DDA algorithm using C and graphics functions in Turbo C++.

Lab 2

Objective: To draw a line between two points using Bresenham's algorithm.

Theory

Bresenham's Line Drawing Algorithm is an efficient method for drawing a straight line between two points in a raster display using only integer calculations. It avoids floating-point arithmetic by using a decision parameter to determine the next pixel, making it faster and more suitable for digital displays. The algorithm incrementally plots the nearest pixel to the theoretical line path, ensuring minimal deviation. It works best when the line slope is between 0 and 1, but it can be adapted for all octants.

Algorithm

Step 1: Input the coordinates (x1, y1) and (x2, y2).

Step 2: Calculate $dx = \text{abs}(x2 - x1)$ and $dy = \text{abs}(y2 - y1)$.

Step 3: Initialize the decision parameter $p = 2*dy - dx$.

Step 4: Set starting point (x, y) = (x1, y1).

Step 5: Plot pixel at (x, y).

Step 6: For x from x1 to x2:

- If $p < 0$, update $p = p + 2*dy$.
- Else, update $p = p + 2*dy - 2*dx$ and increment y by 1.
- Increment x by 1.
- Plot pixel at (x, y).

Step 7: Stop

Program

```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
void main() {
    int gd = DETECT, gm;
    int x1, y1, x2, y2;
    int dx, dy, p, x, y;

    initgraph(&gd, &gm, "C:\\Turboc3\\Projects");

    printf("Enter x1 and y1: ");
    scanf("%d%d", &x1, &y1);
    printf("Enter x2 and y2: ");
    scanf("%d%d", &x2, &y2);
    dx = abs(x2 - x1);
    dy = abs(y2 - y1);

    p = 2 * dy - dx;
```

```

// Swap points if needed to ensure x1 < x2
if (x1 > x2) {
    int tempX = x1, tempY = y1;
    x1 = x2; y1 = y2;
    x2 = tempX; y2 = tempY;
}

x = x1;
y = y1;

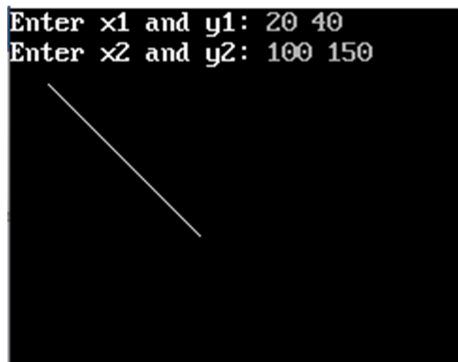
putpixel(x, y, WHITE);

while (x < x2) {
    x++;
    if (p < 0)
        p = p + 2 * dy;
    else {
        y++;
        p = p + 2 * dy - 2 * dx;
    }
    putpixel(x, y, WHITE);
    delay(10);
}

getch();
closegraph();
}

```

Output



Conclusion

Bresenham's Line Drawing Algorithm provides an efficient way to draw straight lines using only integer arithmetic, making it ideal for raster-based displays. This experiment successfully demonstrated how the algorithm minimizes computational overhead while accurately plotting a line between two points, making it a preferred choice in graphics applications.

Lab 3

Objective: To draw a circle on the screen using the Midpoint Circle Drawing Algorithm.

Theory

The Midpoint Circle Drawing Algorithm is a rasterization algorithm used to draw a circle by calculating pixel positions in one-eighth of the circle and reflecting them across the other seven octants. It uses a decision parameter to determine the next pixel, allowing for fast and accurate circle generation using only integer operations.

Algorithm

Step 1: Input center coordinates (xc, yc) and radius r.

Step 2: Initialize (x, y) = (0, r).

Step 3: Set the initial decision parameter $p = 1 - r$.

Step 4: Plot points for all eight symmetrical positions of (x, y).

Step 5: While $x < y$:

- Increment x by 1.
- If $p < 0$:
 - $p = p + 2 * x + 1$
- Else:
 - Decrement y by 1
 - $p = p + 2 * (x - y) + 1$
- Plot points for all eight symmetrical positions of (x, y).

Step 6: Stop when $x \geq y$.

Program

```
#include <graphics.h>

#include <conio.h>

#include <stdio.h>

void plotPoints(int xc, int yc, int x, int y) {

    putpixel(xc + x, yc + y, WHITE);

    putpixel(xc - x, yc + y, WHITE);

    putpixel(xc + x, yc - y, WHITE);

    putpixel(xc - x, yc - y, WHITE);

    putpixel(xc + y, yc + x, WHITE);

    putpixel(xc - y, yc + x, WHITE);

    putpixel(xc + y, yc - x, WHITE);
```

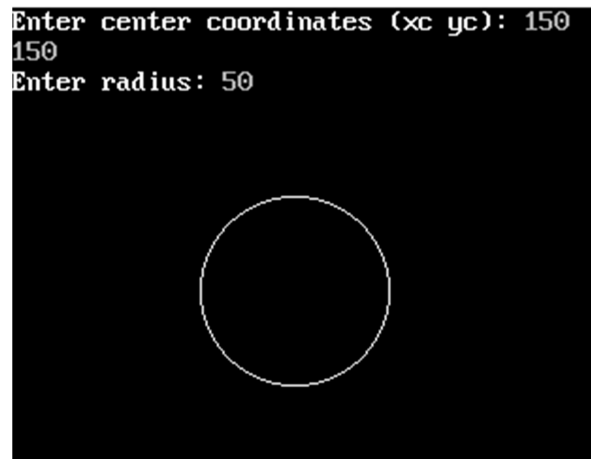
```

        putpixel(xc - y, yc - x, WHITE);
    }
void drawCircle(int xc, int yc, int r) {
    int x = 0, y = r;
    int p = 1 - r;
    plotPoints(xc, yc, x, y);
    while (x < y) {
        x++;
        if (p < 0) {
            p += 2 * x + 1;
        } else {
            y--;
            p += 2 * (x - y) + 1;
        }
        plotPoints(xc, yc, x, y);
        delay(20);
    }
}

void main() {
    int gd = DETECT, gm;
    int xc, yc, r;
    initgraph(&gd, &gm, "C:\\Turboc3\\Projects");
    printf("Enter center coordinates (xc yc): ");
    scanf("%d%d", &xc, &yc);
    printf("Enter radius: ");
    scanf("%d", &r);
    drawCircle(xc, yc, r);
    getch();
    closegraph();
}

```

Output



Conclusion

The Midpoint Circle Drawing Algorithm efficiently plots a circle using symmetry and integer arithmetic. The experiment successfully demonstrated how to generate a smooth circle with minimal calculations.

Lab 4

Objective: To write a program to translate a given graphical object.

Theory

Translation is one of the basic geometric transformations in computer graphics. It involves shifting every point of an object by the same amount in a given direction. If the translation vector is (tx, ty) , then a point (x, y) of the object is translated to $(x + tx, y + ty)$. Translation preserves the shape and size of the object and only changes its position on the screen.

Algorithm

Step 1: Initialize graphics

Step 2: Define original shape coordinates

Step 3: Draw original object

Step 4: Get translation values (tx, ty)

Step 5: Calculate new positions:

$$x_new = x_old + tx$$

$$y_new = y_old + ty$$

Step 6: Draw translated object

Step 7: Close graphics.

Program

```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>

void main() {
    int gd = DETECT, gm;

    int left, top, right, bottom;

    int tx, ty;

    int newLeft, newTop, newRight, newBottom;

    left = 100;
    top = 100;
    right = 200;
    bottom = 150;
```

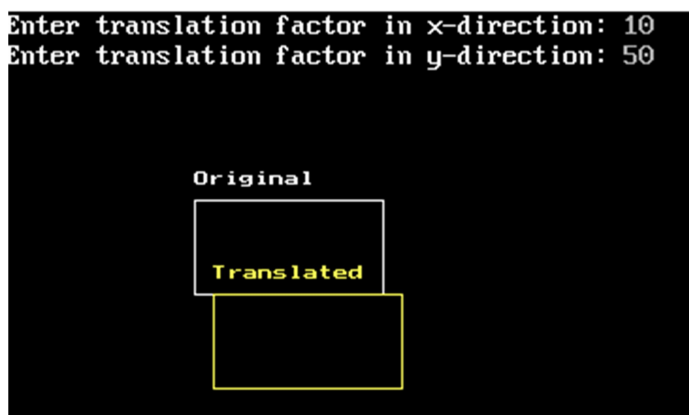
```

initgraph(&gd, &gm, "C:\\Turboc3\\Projects");
printf("Enter translation factor in x-direction: ");
scanf("%d", &tx);
printf("Enter translation factor in y-direction: ");
scanf("%d", &ty);

setcolor(WHITE);
rectangle(left, top, right, bottom);
outtextxy(left, top - 15, "Original");
newLeft = left + tx;
newTop = top + ty;
newRight = right + tx;
newBottom = bottom + ty;
setcolor(YELLOW);
rectangle(newLeft, newTop, newRight, newBottom);
outtextxy(newLeft, newTop - 15, "Translated");
getch();
closegraph();
}

```

Output



Conclusion

The translation transformation effectively moves the graphical object to a new position on the screen without changing its size or shape. This experiment demonstrated how to shift an object by specified offsets using simple coordinate addition.

Lab 5

Objective: To write a program to rotate any object by a specified angle.

Theory

Rotation in computer graphics is a transformation that turns a graphical object around a fixed point (usually the origin) through a specified angle.

Rotation is counter-clockwise for positive angles and clockwise for negative angles. If rotation is done around a fixed point (h,k), then the object is first translated to the origin, rotated, and then translated back.

Algorithm

Step 1: Start the program.

Step 2: Initialize graphics using initgraph().

Step 3: Define coordinates of the object (e.g., triangle or square).

Step 4: Input rotation angle θ from the user.

Step 5: Convert angle from degrees to radians.

Step 6: For each point (x, y), apply the rotation formula:

$$x' = x * \cos\theta - y * \sin\theta$$

$$y' = x * \sin\theta + y * \cos\theta$$

Step 7: Draw original object using line().

Step 8: Draw rotated object using new coordinates.

Step 9: Wait for key press and close graphics window.

Step 10: End.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <math.h>
```

```
#include <stdio.h>
```

```
void main() {
```

```
    int gd = DETECT, gm;
```

```
    int i;
```

```
    int x[3], y[3];
```

```
    int xr[3], yr[3];
```

```
    float angle, rad;
```

```

float cx, cy;

float temp_x, temp_y;


initgraph(&gd, &gm, "C:\\Turboc3\\Projects");


x[0] = 100; y[0] = 100;
x[1] = 150; y[1] = 200;
x[2] = 100; y[2] = 150;


printf("Enter rotation angle in degrees: ");
scanf("%f", &angle);


rad = angle * 3.14159 / 180;


cx = (x[0] + x[1] + x[2]) / 3.0;
cy = (y[0] + y[1] + y[2]) / 3.0;


setcolor(WHITE);
for (i = 0; i < 3; i++) {
    int next = (i + 1) % 3;
    line(x[i], y[i], x[next], y[next]);
}

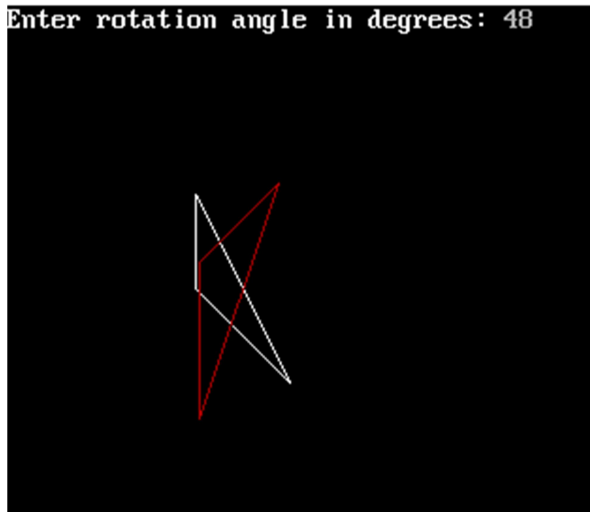

for (i = 0; i < 3; i++) {
    temp_x = x[i] - cx;
    temp_y = y[i] - cy;

    xr[i] = (int)(temp_x * cos(rad) - temp_y * sin(rad) + cx);
    yr[i] = (int)(temp_x * sin(rad) + temp_y * cos(rad) + cy);
}

```

```
setcolor(RED);  
for (i = 0; i < 3; i++) {  
    int next = (i + 1) % 3;  
    line(xr[i], yr[i], xr[next], yr[next]);  
}  
  
getch();  
closegraph();  
}
```

Output



Conclusion

This experiment successfully demonstrated how to apply rotation transformation to a graphical object using trigonometric functions. Rotation is a key concept in graphics, used in animations, modeling, and object transformations.

Lab 6

Objective: To perform composite transformations including translation, rotation, and scaling on a given graphical object.

Theory

Composite transformation refers to the process of applying more than one transformation (like translation, rotation, and scaling) to a graphical object in a specified order. Instead of transforming each point separately for every operation, transformations are combined to achieve the final effect efficiently.

Algorithm

Step 1: Start the program.

Step 2: Initialize graphics mode using `initgraph()`.

Step 3: Define the coordinates of the object (e.g., triangle).

Step 4: Input translation values (t_x , t_y), rotation angle (θ), and scaling factors (s_x , s_y).

Step 5: Apply translation to shift the object.

Step 6: Apply rotation using the formulas:

$$x' = x * \cos\theta - y * \sin\theta$$

$$y' = x * \sin\theta + y * \cos\theta$$

Step 7: Apply scaling using:

$$x'' = x' * s_x$$

$$y'' = y' * s_y$$

Step 8: Draw the original and transformed object.

Step 9: Wait for user input and close the graphics window.

Step 10: End.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <math.h>
```

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
void main() {
```

```
    int gd = DETECT, gm;
```

```
    int i;
```

```
    int x[3] = {100, 150, 100};
```

```

int y[3] = {100, 100, 150};

int xt[3], yt[3];

float angle, sx, sy;

int tx, ty;

float rad;

float cx, cy;

float temp_x, temp_y;

float xr, yr;


initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");


printf("Enter rotation angle in degrees: ");
scanf("%f", &angle);

printf("Enter scaling factors sx and sy: ");
scanf("%f%f", &sx, &sy);

printf("Enter translation values tx and ty: ");
scanf("%d%d", &tx, &ty);


rad = angle * PI / 180.0;

cx = (x[0] + x[1] + x[2]) / 3.0;
cy = (y[0] + y[1] + y[2]) / 3.0;

setcolor(WHITE);

for (i = 0; i < 3; i++) {
    int next = (i + 1) % 3;
    line(x[i], y[i], x[next], y[next]);
}

for (i = 0; i < 3; i++) {
    temp_x = x[i] - cx;
    temp_y = y[i] - cy;

    temp_x = temp_x * sx;

```

```

temp_y = temp_y * sy;

xr = temp_x * cos(rad) - temp_y * sin(rad);
yr = temp_x * sin(rad) + temp_y * cos(rad);

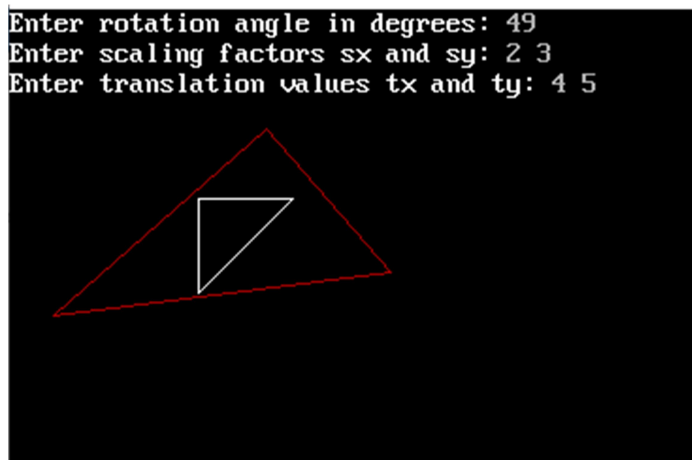
xt[i] = (int)(xr + cx + tx);
yt[i] = (int)(yr + cy + ty);
}

setcolor(RED);
for (i = 0; i < 3; i++) {
    int next = (i + 1) % 3;
    line(xt[i], yt[i], xt[next], yt[next]);
}

getch();
closegraph();
}

```

Output



Conclusion

This experiment demonstrated how multiple transformations can be combined and applied to a graphical object. Composite transformations are essential in graphics to perform complex movements and reshaping of objects efficiently.

Lab 7

Objective: To implement the concept of scaling transformation of a graphical object by given scale factor.

Theory

Scaling is a transformation that changes the size of a graphical object. It enlarges or shrinks the object with respect to the origin or a fixed point. The shape remains the same, but the dimensions change.

For a point (x,y) scaled by factors s_x (in x-direction) and s_y (in y-direction), the new coordinates (x',y') are:

$$x' = x \cdot s_x, y' = y \cdot s_y$$

If both scale factors are greater than 1, the object is enlarged. If they are between 0 and 1, the object is reduced.

Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices of the object.

Step 3: Input the coordinates (x, y) of each vertex.

Step 4: Input the scaling factors s_x and s_y .

Step 5: For each vertex, compute the scaled coordinates:

$$x' = x \times s_x, y' = y \times s_y$$

Step 6: Plot the original object.

Step 7: Plot the scaled object using the new coordinates.

Step 8: End the program.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
void drawPolygon(int x[], int y[], int n, int color) {
```

```
    int i, next;
```

```
    setcolor(color);
```

```
    for (i = 0; i < n; i++) {
```

```
        next = (i + 1) % n;
```

```

        line(x[i], y[i], x[next], y[next]);
    }
}

void main() {
    int gd = DETECT, gm;
    int n, i;
    float sx, sy;
    int x[MAX], y[MAX];
    int x_scaled[MAX], y_scaled[MAX];
    float cx = 0, cy = 0;
    float temp_x, temp_y;

    printf("Enter number of vertices (max %d): ", MAX);
    scanf("%d", &n);

    if (n > MAX) {
        printf("Too many vertices! Maximum allowed is %d.\n", MAX);
        return;
    }

    printf("Enter the coordinates of vertices:\n");
    for (i = 0; i < n; i++) {
        printf("Vertex %d (x y): ", i + 1);
        scanf("%d%d", &x[i], &y[i]);
        cx += x[i];
        cy += y[i];
    }
    cx /= n;
    cy /= n;

    printf("Enter scaling factors sx and sy: ");
    scanf("%f%f", &sx, &sy);

```

```

initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");

// Apply scaling about centroid
for (i = 0; i < n; i++) {
    temp_x = x[i] - cx;
    temp_y = y[i] - cy;

    temp_x *= sx;
    temp_y *= sy;
    x_scaled[i] = (int)(temp_x + cx);
    y_scaled[i] = (int)(temp_y + cy);
}

drawPolygon(x, y, n, WHITE);

drawPolygon(x_scaled, y_scaled, n, RED);

getch();
closegraph();
}

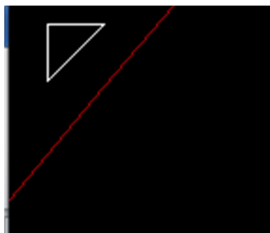
```

Output

```

Enter number of vertices (max 20): 4
Enter the coordinates of vertices:
Vertex 1 (x y): 20 40
Vertex 2 (x y): 20 10
Vertex 3 (x y): 50 10
Vertex 4 (x y): 35 25
Enter scaling factors sx and sy: 0.5 0.6

```



Conclusion

This experiment successfully demonstrated how scaling can change the size of a graphical object while maintaining its shape. Scaling is useful in resizing images, zooming, and various graphical transformations.

Lab 8

Objective: To reflect a graphical object by the X-axis

Theory

Reflection is a transformation that produces a mirror image of an object over a specific axis.

When an object is reflected over the X-axis, its X-coordinates remain the same, but Y-coordinates are inverted.

For a point (x,y), the reflected point over the X-axis becomes: $(x',y')=(x,-y)$

This flips the object vertically, creating a mirror image across the X-axis.

Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices of the object.

Step 3: Input the coordinates (x, y) of each vertex.

Step 4: For each vertex, compute the reflected coordinates:

$$x' = x$$

$$y' = -y$$

Step 5: Plot the original object.

Step 6: Plot the reflected object using the new coordinates.

Step 7: End the program.

Program

```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>

void drawPolygon(int x[], int y[], int n, int color) {
    int i;
    setcolor(color);
    for (i = 0; i < n; i++) {
        int next = (i + 1) % n;
        line(x[i], y[i], x[next], y[next]);
    }
}

void main() {
    int gd = DETECT, gm;
    int n, i;
    int x[20], y[20];
```

```

int x_reflected[20], y_reflected[20];

printf("Enter number of vertices: ");

scanf("%d", &n);

printf("Enter coordinates of vertices:\n");

for (i = 0; i < n; i++) {

    printf("Vertex %d (x y): ", i + 1);

    scanf("%d%d", &x[i], &y[i]);

}

initgraph(&gd, &gm, "C:\\Turboc3\\Projects");

for (i = 0; i < n; i++) {

    x_reflected[i] = x[i];

    y_reflected[i] = getmaxy() - y[i];

}

drawPolygon(x, y, n, WHITE);

drawPolygon(x_reflected, y_reflected, n, RED);

getch();

closegraph();

}

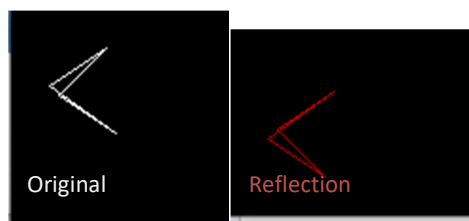
```

Output

```

Enter number of vertices: 4
Enter coordinates of vertices:
Vertex 1 (x y): 20 40
Vertex 2 (x y): 50 20
Vertex 3 (x y): 25 45
Vertex 4 (x y): 55 65_

```



Conclusion

This experiment successfully demonstrated how to reflect a graphical object over the X-axis using coordinate transformation. It helped visualize how reflection flips an object vertically while preserving its shape.

Lab 9

Objective: To reflect a graphical object across the Y-axis.

Theory

Reflection is a geometric transformation that produces a mirror image of an object about a specified axis. When an object is reflected over the Y-axis, the x-coordinates of its points change sign while the y-coordinates remain the same.

The transformation formulas for reflection across the Y-axis are:

$$x' = -x, y' = y \quad \text{or} \quad y' = y, x' = -x$$

This flips the object horizontally, producing a mirror image on the other side of the Y-axis.

Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices of the polygon.

Step 3: Input the coordinates of each vertex (x, y).

Step 4: For each vertex, apply Y-axis reflection:

$$x' = -x$$

$$y' = y$$

Step 5: Plot the original object.

Step 6: Plot the reflected object.

Step 7: End.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
void drawPolygon(int x[], int y[], int n, int color) {
```

```
    int i, next;
```

```
    setcolor(color);
```

```
    for (i = 0; i < n; i++) {
```

```
        next = (i + 1) % n;
```

```
        line(x[i], y[i], x[next], y[next]);
```

```
    }
```

```
}
```

```

void main() {
    int gd = DETECT, gm;
    int i, n;
    int x[MAX], y[MAX];
    int x_reflected[MAX], y_reflected[MAX];
    int midX;

    printf("Enter number of vertices (max %d): ", MAX);
    scanf("%d", &n);

    if (n < 3 || n > MAX) {
        printf("Invalid number of vertices.\n");
        return;
    }

    printf("Enter coordinates of the polygon vertices:\n");
    for (i = 0; i < n; i++) {
        printf("Vertex %d (x y): ", i + 1);
        scanf("%d%d", &x[i], &y[i]);
    }

    initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");

    midX = getmaxx() / 2;

    for (i = 0; i < n; i++) {
        x[i] = x[i] + midX;
    }

    setcolor(LIGHTGRAY);
    line(midX, 0, midX, getmaxy());
}

```

```

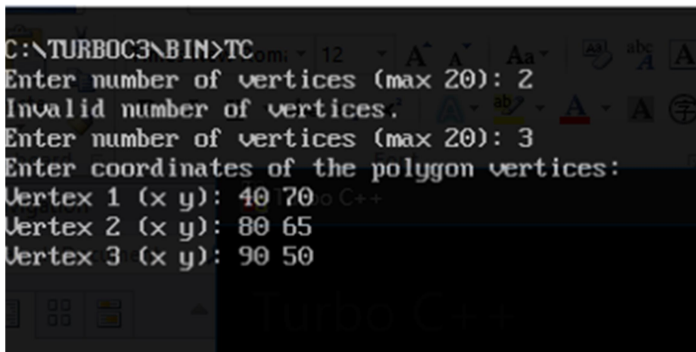
for (i = 0; i < n; i++) {
    x_reflected[i] = midX - (x[i] - midX);
    y_reflected[i] = y[i];
}

drawPolygon(x, y, n, WHITE);
drawPolygon(x_reflected, y_reflected, n, RED);

setcolor(WHITE);
outtextxy(x[0], y[0] - 15, "Original");
setcolor(RED);
outtextxy(x_reflected[0], y_reflected[0] - 15, "Reflected");
getch();
closegraph();
}

```

Output



Conclusion

Reflection over the Y-axis is achieved by negating the x-coordinate of each point. This lab proved that geometric transformations like reflection can be easily visualized and implemented in 2D graphics programming.

Lab 10

Objective: To apply the X-axis shearing to a graphical object.

Theory

Shearing is a geometric transformation that distorts the shape of an object by shifting its coordinates along one axis. When shearing is done along the X-axis, each point's x-coordinate is shifted by an amount proportional to its y-coordinate, while the y-coordinate remains unchanged.

The formula for X-axis shearing is:

$$x' = x + shx \times y \quad y' = y$$

Where:

- (x, y) is the original point
- (x', y') is the sheared point
- shx is the shearing factor along the X-axis

Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices of the polygon.

Step 3: Input the coordinates (x, y) of each vertex.

Step 4: Input the X-axis shearing factor shx.

Step 5: For each point, apply X-axis shear:

$$x' = x + shx \times y$$

$$y' = y$$

Step 6: Draw the original object.

Step 7: Draw the sheared object.

Step 8: Stop.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
void drawPolygon(int x[], int y[], int n, int color) {
```

```
    int i, next;
```

```
    setcolor(color);
```

```
    for (i = 0; i < n; i++) {
```

```
        next = (i + 1) % n;
```

```

        line(x[i], y[i], x[next], y[next]);
    }
}

void main() {
    int gd = DETECT, gm;
    int n, i;
    float shx;
    int x[MAX], y[MAX];
    int x_sheared[MAX], y_sheared[MAX];
    int shiftX;

    printf("Enter number of vertices (max %d): ", MAX);
    scanf("%d", &n);

    if (n < 3 || n > MAX) {
        printf("Invalid number of vertices.\n");
        return;
    }

    printf("Enter coordinates of the polygon vertices:\n");
    for (i = 0; i < n; i++) {
        printf("Vertex %d (x y): ", i + 1);
        scanf("%d%d", &x[i], &y[i]);
    }

    printf("Enter shearing factor along X-axis (shx): ");
    scanf("%f", &shx);

    initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");

    shiftX = 100;

    for (i = 0; i < n; i++) {
        x[i] = x[i] + shiftX;

```

```

    }

    for (i = 0; i < n; i++) {
        x_sheared[i] = (int)(x[i] + shx * y[i]);
        y_sheared[i] = y[i];
    }

    drawPolygon(x, y, n, WHITE);

    drawPolygon(x_sheared, y_sheared, n, RED);

    setcolor(WHITE);

    outtextxy(x[0], y[0] - 15, "Original");

    setcolor(RED);

    outtextxy(x_sheared[0], y_sheared[0] - 15, "Sheared");

    getch();

    closegraph();
}

```

Output

```

C:\TURBOC3\BIN>TC
Enter number of vertices (max 20): 3
Enter coordinates of the polygon vertices:
Vertex 1 (x y): 60 60
Vertex 2 (x y): 50 50
Vertex 3 (x y): 80 80
Enter shearing factor along X-axis (shx): 1

```



Conclusion

This experiment demonstrated how X-axis shearing affects a graphical object by slanting it horizontally. The object's shape gets distorted depending on the shear factor and height, while the vertical dimensions remain unchanged. This transformation is useful in simulating perspective and performing shape manipulation in graphics.

Lab 11

Objective: To apply Y-axis shearing transformation to an object.

Theory

Shearing is a geometric transformation that distorts the shape of an object by shifting its points in one axis direction relative to another axis. In Y-axis shearing, the X-coordinates remain unchanged, while the Y-coordinates are shifted based on their X-values.

The transformation formulas for Y-shearing are: $x' = x$, $y' = y + shy \cdot x$

Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices of the object.

Step 3: Input the coordinates (x, y) of each vertex.

Step 4: Input the shearing factor shy.

Step 5: For each point, apply Y-shear using:

$$x' = x$$

$$y' = y + shy \times x$$

Step 6: Draw the original object.

Step 7: Draw the sheared object.

Step 8: End the program

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
void drawPolygon(int x[], int y[], int n, int color) {  
    int i, next;  
    setcolor(color);  
    for (i = 0; i < n; i++) {  
        next = (i + 1) % n;  
        line(x[i], y[i], x[next], y[next]);  
    }  
}
```

```
}
```

```
void main() {  
    int gd = DETECT, gm;  
  
    int n, i;  
  
    float shy;  
  
    int x[MAX], y[MAX];  
  
    int x_sheared[MAX], y_sheared[MAX];  
  
    int shiftX = 100;  
  
    printf("Enter number of vertices (max %d): ", MAX);  
    scanf("%d", &n);  
  
    if (n < 3 || n > MAX) {  
        printf("Invalid number of vertices.\n");  
        return;  
    }  
  
    printf("Enter coordinates of the polygon vertices:\n");  
    for (i = 0; i < n; i++) {  
        printf("Vertex %d (x y): ", i + 1);  
        scanf("%d%d", &x[i], &y[i]);  
    }  
  
    printf("Enter shearing factor along Y-axis (shy): ");  
    scanf("%f", &shy);  
  
    for (i = 0; i < n; i++) {  
        x[i] = x[i] + shiftX;  
    }  
  
    initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");  
  
    for (i = 0; i < n; i++) {
```

```

    x_sheared[i] = x[i];
    y_sheared[i] = (int)(y[i] + shy * x[i]);
}

drawPolygon(x, y, n, WHITE);

drawPolygon(x_sheared, y_sheared, n, RED);

setcolor(WHITE);
outtextxy(x[0], y[0] - 15, "Original");

setcolor(RED);
outtextxy(x_sheared[0], y_sheared[0] - 15, "Sheared");

getch();
closegraph();

```

Output

```

Enter number of vertices (max 20): 4
Enter coordinates of the polygon vertices:
Vertex 1 (x y): 20 40
Vertex 2 (x y): 10 20
Vertex 3 (x y): 30 25
Vertex 4 (x y): 40 30
Enter shearing factor along Y-axis (shy): 0.4

```



Conclusion

This experiment successfully demonstrated Y-axis shearing, where the object is vertically slanted while maintaining its original width. Such transformations are useful in creating skewed shapes, simulating motion, and applying perspective in computer graphics.

Lab 12

Objective: To implement and visualize 3D transformation with different parameters

Theory

3D transformation involves modifying the position, size, or orientation of objects in a three-dimensional space using transformation matrices. These transformations are essential for modeling, animation, and rendering in computer graphics.

Since standard 2D graphics libraries (like Turbo C) do not support direct 3D drawing, 3D objects are projected into 2D space using a simple projection technique (such as orthographic or perspective projection).

Algorithm

Step 1: Define a 3D object (e.g., cube) with its vertices.

Step 2: Input the transformation type and parameters (tx, ty, tz, sx, sy, sz, angle).

Step 3: Apply the transformation formulas to each vertex.

Step 4: Use a simple projection (like orthographic or isometric) to convert (x, y, z) to (X, Y) on screen.

Step 5: Draw lines between the transformed points to represent the object.

Step 6: Show both original and transformed object for comparison.

Step 7: End the program.

Program

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
void project3Dto2D(float x, float y, float z, int *px, int *py) {  
    *px = (int)(x + 200);  
    *py = (int)(y + 150);  
}
```

```
void drawCube(int p[8][2], int color) {  
    int i;
```

```

setcolor(color);

for (i = 0; i < 4; i++)
    line(p[i][0], p[i][1], p[(i + 1) % 4][0], p[(i + 1) % 4][1]);

for (i = 0; i < 4; i++)
    line(p[i + 4][0], p[i + 4][1], p[4 + (i + 1) % 4][0], p[4 + (i + 1) % 4][1]);

for (i = 0; i < 4; i++)
    line(p[i][0], p[i][1], p[i + 4][0], p[i + 4][1]);
}

void main() {
    int gd = DETECT, gm;

    int i;

    float tx, ty, tz;

    float sx, sy, sz;

    float angle, rad;

    float x, y, z;

    float x_rot, y_rot;

    float cube[8][3] = {
        {0, 0, 0}, {100, 0, 0}, {100, 100, 0}, {0, 100, 0},
        {0, 0, 100}, {100, 0, 100}, {100, 100, 100}, {0, 100, 100}
    };

    float transformed[8][3];

    int projected[8][2];

    int orig_proj[8][2];

```

```

printf("Enter rotation angle (Z-axis) in degrees: ");
scanf("%f", &angle);

printf("Enter scaling factors (sx sy sz): ");
scanf("%f%f%f", &sx, &sy, &sz);

printf("Enter translation values (tx ty tz): ");
scanf("%f%f%f", &tx, &ty, &tz);


rad = angle * 3.1416 / 180.0;


initgraph(&gd, &gm, "C:\\TURBOC3\\Projects");


for (i = 0; i < 8; i++) {
    project3Dto2D(cube[i][0], cube[i][1], cube[i][2],
        &orig_proj[i][0], &orig_proj[i][1]);
}

drawCube(orig_proj, WHITE);

outtextxy(orig_proj[0][0], orig_proj[0][1] - 10, "Original");


for (i = 0; i < 8; i++) {
    x = cube[i][0];
    y = cube[i][1];
    z = cube[i][2];

    x = x * sx;
    y = y * sy;
    z = z * sz;

    x_rot = x * cos(rad) - y * sin(rad);
    y_rot = x * sin(rad) + y * cos(rad);

    // Translate
    transformed[i][0] = x_rot + tx;
    transformed[i][1] = y_rot + ty;
    transformed[i][2] = z + tz;

```

```

// Project to 2D

project3Dto2D(transformed[i][0], transformed[i][1], transformed[i][2],
               &projected[i][0], &projected[i][1]);
}

drawCube(projected, RED);

outtextxy(projected[0][0], projected[0][1] - 10, "Transformed");

getch();

closegraph();
}

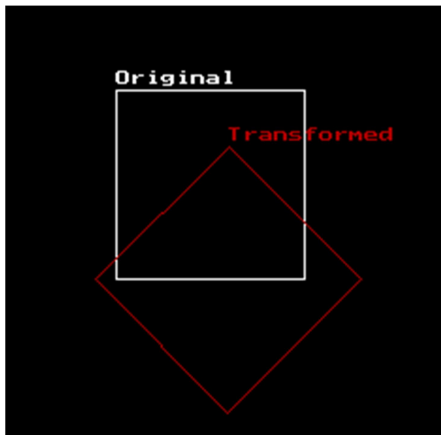
```

Output

```

C:\TURBOC3\BIN>TC
Enter rotation angle (Z-axis) in degrees: 45
Enter scaling factors (sx sy sz): 1 1 1
Enter translation values (tx ty tz): 60 30 40_

```



Conclusion

This experiment demonstrated how 3D objects can be manipulated using transformations such as translation, rotation, and scaling. Though displayed on a 2D screen, the use of projections helped visualize these changes, which are fundamental in 3D modeling, simulations, and modern graphics applications.